



CSC232 – INFORMATION SECURITY

Lab Assignment 1: Classical Ciphers



Submitted By:
Muhammad Rizwan Shafiq
SP24-BCS-069
BCS-4B

Instructor:
Sir Sabghat Ullah Khan

Assignment 1: Classical Ciphers

Introduction

This assignment was based on Lab 3 from the Information Security Lab Manual. The focus was on implementing and analyzing classical cryptographic ciphers including Affine, Multiplicative, Vigenère, and Transposition ciphers. The objective was to understand encryption, decryption, and cryptanalysis techniques.

Task 1: Affine Cipher

The Affine Cipher combines both multiplicative and additive transformations. Encryption is performed using the formula $C = (A \times P + B) \bmod 26$ and decryption using $P = A^{-1} \times (C - B) \bmod 26$. The implementation ensures correct handling of uppercase letters and non-alphabetic symbols.

Sample Code Snippet:

```
• def affine_encrypt(text, A, B):  
    result = ""  
    for char in text.upper():  
        if char.isalpha():  
            x = ord(char) - ord('A')  
            enc = (A * x + B) % 26  
            result += chr(enc + ord('A'))  
        else:  
            result += char  
    return result
```

Example Output:

Plaintext: AFFINE CIPHER

Encrypted: IHHWVCSWFRCP

Decrypted: AFFINECIPHER

Task 2: Multiplicative Cipher

This cipher uses modular multiplication for encryption and decryption. Only keys coprime with 26 are valid to ensure existence of modular inverse.

Sample Code Snippet:

```
• def multiplicative_encrypt(plaintext, key):  
    ciphertext = ""  
    for char in plaintext.upper():  
        if char.isalpha():  
            num = ord(char) - ord('A')  
            enc = (num * key) % 26  
            ciphertext += chr(enc + ord('A'))
```

```
    else:  
        ciphertext += char  
    return ciphertext
```

Example Output:

Plaintext: HELLO WORLD

Ciphertext: TEBBW FQBPF

Decrypted: HELLO WORLD

Task 3: Vigenère Cipher (CrypTool 2)

CrypTool 2 was used to experiment with the Vigenère cipher. The process included encrypting a sample plaintext using a key, performing frequency and Kasiski analysis, and recovering the key using cryptanalysis tools. The results confirmed correct encryption and decryption, with visual frequency analysis confirming the non-random pattern of ciphertext.

Task 4: Transposition Cipher

A transposition cipher rearranges characters according to a key sequence. This implementation included dynamic key generation, handling of spaces and punctuation, and padding for alignment. The key length was capped at 9 digits for simplicity.

Sample Code Snippet:

- def encode(key, plaintext):
 key_len = len(key)
 order = {int(val): num for num, val in enumerate(key)}
 ciphertext = ""
 for index in sorted(order.keys()):
 for part in split_len(plaintext, key_len):
 try:
 ciphertext += part[order[index]]
 except IndexError:
 pass
 return ciphertext

Instructor Comment: Key length handling capped at 9 digits restricts longer messages; spaces/punctuation preserved but alignment may break with padding.

Summary

Through this assignment, various classical ciphers were implemented and analyzed. The exercise enhanced understanding of modular arithmetic, encryption/decryption processes, and basic cryptanalysis. Future improvements include support for longer keys and better text alignment.